

GUIA DEL PROYECTO – CIVILIZATIONS

Proyecto AMS / AWS – Curso 2025–2026

Centro: Institut Esteve Terradas i illa

Autores:

- Ariadna Martinez
- Anna Puig
- Melisbeth Rojas

MANUAL DEL PROYECTO – CIVILIZATIONS.....	1
1. Introducción.....	2
2. Análisis y diseño.....	3
Pantallas principales.....	3
Estados del juego.....	3
Escenas.....	4
Personajes principales.....	4
Mecánicas del juego.....	4
Diagrama de clases UML.....	4
Diagrama de secuencia (si aplica).....	5
3. Arquitectura y desarrollo.....	5
TimerTasks.....	6
4. Test y depuración (Anna).....	6
5. Recursos gráficos y de audio.....	6
6. Manual de usuario.....	6
Gestión de recursos.....	7
Construcción de edificios.....	7
Entrenamiento de unidades.....	7
Sistema de amenazas.....	7
Batallas.....	8
Botones generales.....	8
7. Reflexión y mejoras.....	8

1. Introducción

Civilizations es un proyecto de desarrollo de un videojuego de estrategia y gestión en el que el jugador controla una civilización, administra recursos, construye edificios, entrena unidades militares y se enfrenta a amenazas enemigas. El juego combina mecánicas de gestión, automatización y combate, integrando contenidos de varios módulos del ciclo formativo, especialmente (falta poner los módulos).

El proyecto se ha desarrollado en equipo utilizando metodologías de trabajo colaborativo y herramientas de control de versiones, garantizando una estructura organizada y un flujo de trabajo eficiente.

1.1. Descripción del proyecto

Civilizations es un videojuego en el que el jugador debe mantener y hacer crecer una civilización mediante la gestión de recursos, la construcción de edificios y la creación de unidades militares.

El sistema incorpora:

- Generación automática de recursos mediante procesos temporizados
- Construcción de edificios con diferentes funciones
- Entrenamiento de unidades militares
- Sistema de batallas contra enemigos
- Amenazas automáticas que aparecen durante la partida
- Interfaz gráfica de usuario (GUI) desarrollada en Java

El objetivo principal es sobrevivir, expandirse y defender la civilización ante ataques enemigos.

1.2. Objetivos del proyecto

- Desarrollar un videojuego funcional aplicando conocimientos de programación orientada a objetos.
- Integrar diferentes módulos del ciclo formativo en un único proyecto completo.
- Practicar el trabajo en equipo utilizando Git y GitHub para el control de versiones.
- Diseñar una interfaz gráfica intuitiva y funcional.
- Implementar automatizaciones mediante TimerTask para simular procesos del juego.
- Aplicar buenas prácticas de arquitectura y organización del código.

1.3. Herramientas utilizadas

- IDE: Eclipse
- Lenguaje: Java, SQL, HTML, CSS, JavaFx
- Control de versiones: Git + GitHub
- Diseño de diagramas: Draw.io
- Edición de documentos: Google Docs, PDF
- Herramientas de diseño: Canva/Figma

2. Análisis y diseño

Esta sección describe los requisitos del proyecto, su funcionamiento interno, la estructura general del juego y los elementos principales que lo componen. También incluye los diagramas necesarios para representar la arquitectura del sistema.

2.1. Especificaciones funcionales

1. Gestión de recursos

El sistema debe permitir:

- Mostrar los recursos actuales de la civilización (comida, madera, hierro, maná).
- Actualizar los recursos automáticamente cada cierto tiempo.
- Consumir recursos al construir edificios o entrenar unidades.
- Impedir acciones si no hay recursos suficientes.

2. Construcción de edificios

El sistema debe permitir:

- Mostrar la lista de edificios disponibles.
- Construir un edificio si el jugador tiene los recursos necesarios.
- Aplicar los efectos del edificio (producción, defensa, mejoras).
- Registrar el edificio construido en el estado de la civilización.

3. Entrenamiento de unidades

El sistema debe permitir:

- Mostrar las unidades disponibles.
- Entrenar unidades si se cumplen los requisitos de recursos.
- Añadir las unidades entrenadas al ejército del jugador.
- Mostrar estadísticas básicas de cada unidad.

4. Sistema de amenaza

El sistema debe:

- Mostrar el nivel actual de amenaza.
- Aumentar la amenaza automáticamente con el paso del tiempo.
- Disminuir la amenaza cuando el jugador gana una batalla manual.
- Generar batallas automáticas cuando la amenaza alcanza ciertos niveles.

5. Batallas automáticas

El sistema debe:

- Ejecutar una batalla automática cada cierto intervalo de tiempo.
- Generar un ejército enemigo en función del nivel de amenaza.
- Resolver la batalla comparando estadísticas de ambos ejércitos.
- Guardar un informe de la última batalla automática.
- Mostrar el resultado al usuario.

6. Batallas manuales

El sistema debe permitir:

- Mostrar el ejército del jugador y el enemigo.
- Resolver la batalla según las estadísticas de las unidades.
- Mostrar un informe detallado del combate.
- Reducir la amenaza si el jugador gana.

7. Interfaz gráfica (GUI)

El sistema debe:

- Mostrar pantallas separadas para recursos, unidades, edificios, tecnologías, defensas y batallas.
- Actualizar la información en tiempo real al abrir cada pantalla.
- Mostrar iconos representativos de unidades, recursos y edificios.
- Permitir navegar entre pantallas mediante botones.

8. Persistencia de datos

El sistema debe permitir:

- Guardar la partida (estado de recursos, unidades, edificios, amenaza).
- Cargar una partida previamente guardada.
- Mantener la coherencia de los datos entre sesiones.

9. Generación de enemigos

El sistema debe:

- Crear ejércitos enemigos en función del nivel de amenaza.
- Variar la composición del ejército enemigo según la dificultad.
- Garantizar que siempre exista un enemigo válido para las batallas.

10. Informes y registros

El sistema debe:

- Mostrar el informe de la última batalla automática.
- Mostrar el informe de la batalla manual recién realizada.
- Registrar los escombros generados (solo visual, no se suman a recursos).
- Mostrar estadísticas relevantes al usuario.

2.2. Especificaciones no funcionales

- **Rendimiento:** la generación automática de recursos debe ejecutarse sin bloquear la interfaz.
- **Usabilidad:** la GUI debe ser clara, intuitiva y fácil de navegar.
- **Escalabilidad:** el sistema debe permitir añadir nuevos edificios o unidades sin modificar toda la estructura.
- **Mantenibilidad:** el código debe estar organizado en paquetes y clases coherentes.
- **Fiabilidad:** los procesos automáticos deben ejecutarse correctamente durante toda la partida.
- **Compatibilidad:** el juego debe funcionar en cualquier sistema con Java instalado.

2.3. Estructura del proyecto (pantallas, estados, escenas)

Pantallas principales

- Pantalla de inicio
- Pantalla principal del juego (recursos, edificios, unidades)
- Pantalla de batalla
- Pantalla de amenazas
- Pantalla de información (HUD)

Estados del juego

- Estado de gestión (recursos y construcción)
- Estado de preparación militar
- Estado de batalla
- Estado de amenaza activa

Escenas

- Escena de gestión
- Escena de combate
- Escena de alerta/amenaza

2.4. Diseño de personajes y mecánicas del proyecto

Personajes principales

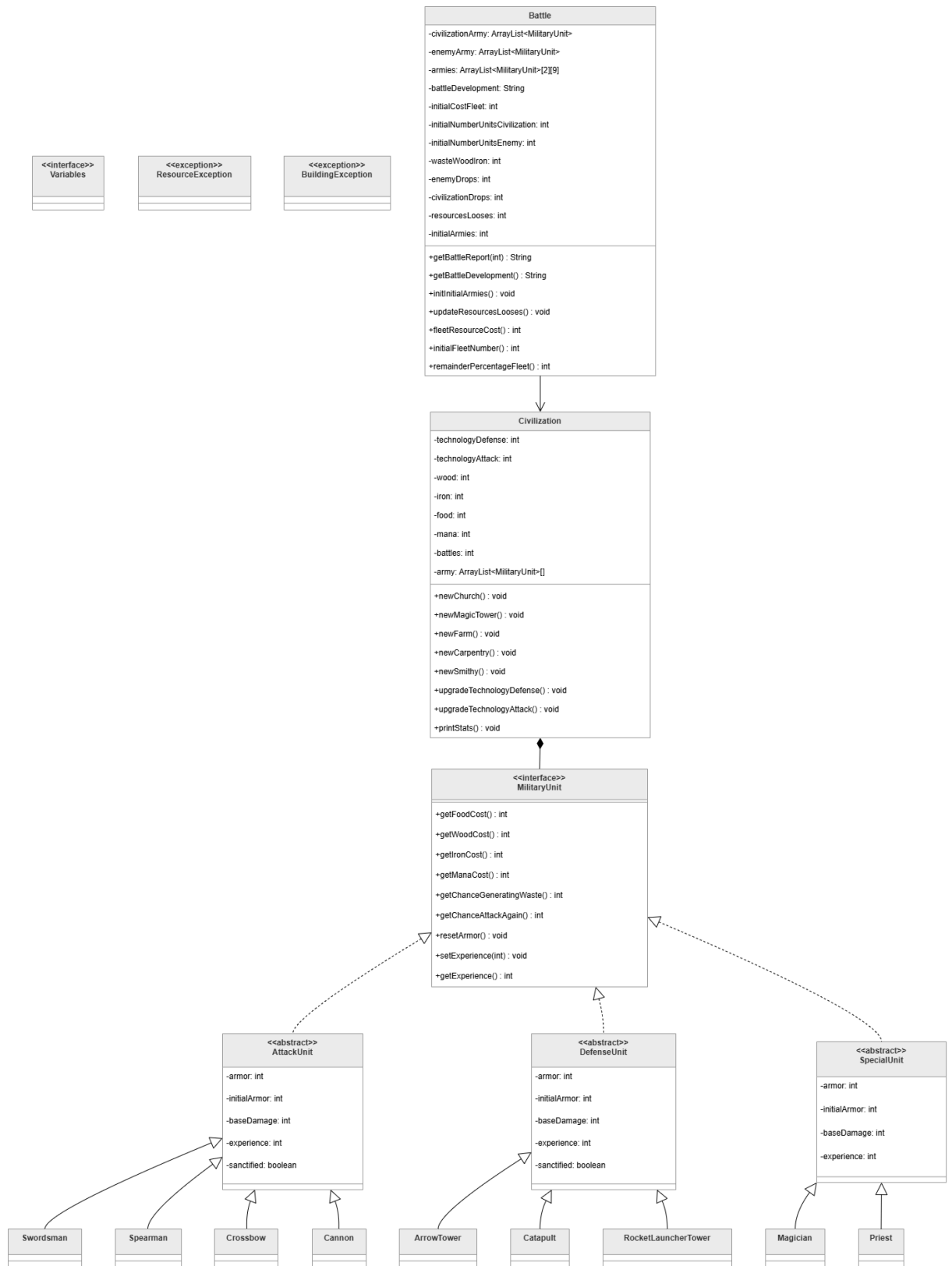
- **Jugador:** controla la civilización, gestiona recursos y toma decisiones.
- **Enemigos:** aparecen automáticamente y atacan la civilización.
- **Unidades militares:** soldados, arqueros, caballería (según el diseño del equipo).
- **Edificios:** granja, mina, cuartel, muralla, etc.

Mecánicas del juego

- Generación automática de recursos.
- Construcción de edificios que desbloquean nuevas funciones.
- Entrenamiento de unidades militares.
- Sistema de combate basado en estadísticas.
- Amenazas automáticas que obligan al jugador a defenderse.
- Progresión de la civilización según las decisiones del jugador.

2.5. Diagramas UML (de clases, de secuencia si corresponde)

Diagrama de clases UML



3. Arquitectura y desarrollo

Esta sección describe cómo está organizado el código del proyecto, cuáles son los componentes principales del juego, cómo se estructura el proyecto internamente y cómo fluye la información durante la ejecución.

3.1. Descripción de la estructura del código fuente

Logic → Lógica del juego

Timers → Procesos automáticos

GUI → Interfaz gráfica

Utils → Funciones auxiliares

3.2. Explicación de los componentes principales:

- Jugador
Representa la civilización controlada por el usuario. Gestiona recursos, construye edificios y entrena unidades.
- Enemigos
Aparecen automáticamente mediante procesos temporizados y atacan la civilización.
- Colisiones
En este proyecto no hay colisiones físicas como en un juego 2D, pero sí hay “colisiones lógicas”:
Unidades vs enemigos
Ataques vs defensas
Recursos vs capacidad de almacenamiento
- proyectiles
En este proyecto no existen proyectiles gráficos, pero sí **acciones de ataque** representadas mediante cálculos de daño.
- HUD (interfaz de información)

La GUI muestra:

- Recursos actuales
- Edificios construidos
- Unidades disponibles
- Alertas de amenazas
- Botones de acción

TimerTasks

Procesos automáticos que ejecutan acciones cada cierto tiempo

- ResourceTimerTask → genera recursos
- AttackTimerTask → lanza ataques enemigos

- 3.3. Diagrama de archivos/proyecto
- 3.4. Flujo de datos y eventos del juego
 - 1. El jugador inicia la partida.
 - 2. La GUI carga los recursos, edificios y unidades.
 - 3. Los **TimerTask** comienzan a ejecutarse:
 - ResourceTimerTask genera recursos.
 - AttackTimerTask crea amenazas.
 - 4. El jugador construye edificios y entrena unidades.
 - 5. Si aparece una amenaza, se activa el sistema de batalla.
 - 6. El resultado de la batalla modifica los recursos y unidades.
 - 7. El ciclo continúa hasta que el jugador gana o pierde.

4. Test y depuración (Anna)

Esta sección recoge todas las pruebas realizadas durante el desarrollo del proyecto, los errores detectados, cómo se solucionaron y las mejoras aplicadas. El objetivo es demostrar que el juego ha sido verificado, corregido y optimizado antes de su entrega final.

4.1. Errores de configuración del entorno y ejecución

- El IDE no encontraba el método main

El proyecto no se ejecutaba porque el package de Main.java estaba mal declarado.

Se corrigió la cabecera y se ejecutó desde Run As → Java Application.

- Configuraciones de ejecución duplicadas

El IDE generaba múltiples configuraciones como "Main (1)", "Main (2)", "Main (3)...".

Se eliminaron las configuraciones duplicadas y se dejó solo la correcta.

- Incompatibilidad entre versión del compilador y JRE

El proyecto compilaba en Java 25 pero se ejecutaba con Java 17.

Se bajó el nivel de compilación a Java 17.

- Error "Unsupported major.minor version 65.0"

El bytecode generado era demasiado nuevo para el JRE instalado.

Se recompiló todo el proyecto en Java 17.

- El IDE no detectaba el JDK

El proyecto usaba un JRE en lugar de un JDK.

Se configuró manualmente el JDK 17 en Installed JREs.

- Problemas con module-info.java

El archivo de módulos bloqueaba la visibilidad entre paquetes.

Se eliminó para trabajar con arquitectura clásica no modular.

- Conflicto con la JRE System Library

Al clonar el proyecto, la librería del sistema aparecía como "Unbound".

Se eliminó y se añadió de nuevo usando Execution Environment.

4.2. Errores de lógica del juego y arquitectura (Backend Java)

- Recursos iniciales a 0

La civilización empezaba sin recursos.

Se añadieron valores iniciales para permitir jugar.

- Falta de persistencia (guardar/cargar partida)

El juego no guardaba el progreso.

Se implementó un sistema de persistencia con JDBC.

- Menú incompleto

No había opciones para guardar o cargar partida.

Se añadieron nuevos casos en el menú principal.

- Falta de claridad en las tareas de Persona 1

Había confusión sobre qué debía programar cada uno.

Se definió una lista clara de responsabilidades.

- Desorganización de paquetes

Todas las clases estaban en la raíz del proyecto.

Se reorganizó en paquetes: Buildings, Units, Logic, Interfaces, Exceptions y Save_Load.

4.3. Errores de integración con la interfaz gráfica (GUI)

- EnemyGenerator no encontrado

El archivo no existía en el paquete esperado, provocando errores de importación.

Se creó el archivo en el paquete correcto y se actualizó la importación en GameState.

- La amenaza permanecía siempre en 0

El temporizador de batallas automáticas no se ejecutaba y initGame() no se llamaba correctamente.

Se verificó la ejecución del método, se corrigió la ubicación de EnemyGenerator y se confirmó el funcionamiento del temporizador.

- La amenaza no aparecía al entrar por primera vez al Desktop

El valor solo se actualizaba al refrescar la vista.

Se añadió la actualización del label en el método initialize().

- La vista de batalla no mostraba unidades

Las listas aparecían vacías por falta del archivo battleUnitItem.fxml y su controlador.

Se crearon ambos archivos, permitiendo mostrar correctamente las unidades.

- La última batalla automática no se mostraba

El informe no aparecía al abrir la vista de batalla.

Se añadió la carga del último informe mediante GameState.getLastAutoBattle().

- Confusión sobre los recursos obtenidos en batalla

Se asumió erróneamente que los escombros debían sumarse a los recursos del jugador.

Se confirmó que solo deben mostrarse en el informe, sin afectar a los recursos.

- La amenaza no bajaba al ganar una batalla manual
El valor aumentaba correctamente, pero no disminuía al ganar.
Se añadió la llamada a `GameState.decreaseThreat(15)` en el controlador de batalla.

4.4. Mejoras aplicadas durante el desarrollo

- **Reorganización completa de la arquitectura del proyecto**

Se estructuraron correctamente los paquetes, separando responsabilidades y facilitando la mantenibilidad del código.

- **Implementación de un sistema de persistencia funcional**

Se añadió guardado y carga de partidas, permitiendo continuar el progreso en sesiones posteriores.

- **Optimización del flujo del juego**

Se ajustaron los valores iniciales de recursos, se mejoró la interacción del menú y se corrigieron errores de lógica interna.

- **Corrección de incompatibilidades del entorno**

Se unificó la versión de Java utilizada por todo el equipo, evitando errores de compilación y ejecución.

- **Actualización correcta de la interfaz gráfica**

Se añadieron actualizaciones en tiempo real para la amenaza, unidades y registros de batalla.

- **Creación y estandarización de componentes FXML**

Se añadieron archivos faltantes como `battleUnitItem.fxml` y sus controladores asociados.

- **Mejoras en la visualización de batallas**

Se implementó la carga automática del último informe de batalla y la visualización correcta de unidades.

- **Clarificación de la lógica de recursos y escombros**

Se ajustó la interpretación del PDF para evitar comportamientos incorrectos en la interfaz.

5. Recursos gráficos y de audio

5.1. Fuentes utilizadas (con derechos de uso si corresponde)

- Iconos e imágenes libres de derechos

Para la interfaz se han utilizado iconos e imágenes procedentes de bancos de recursos gratuitos que permiten su uso en proyectos educativos, tales como:

Flaticon

Los recursos gráficos empleados incluyen iconos representativos de:

Unidades militares (espadachín, lancero, ballestero, mago, sacerdote)

Máquinas de asedio (catapultas, cañones, torres)

Recursos del juego (madera, hierro, comida, maná)

Todos los recursos se han utilizado exclusivamente con fines educativos dentro del marco del proyecto.

5.2. Cómo se han integrado en el proyecto

- Organización de archivos

Los recursos gráficos se han almacenado siguiendo la estructura estándar de JavaFX, manteniendo una organización clara y accesible:

Carpeta de iconos:

`src/main/resources/assets/icons/`

Componentes visuales reutilizables (como las plantillas de unidades en batalla):

`src/main/resources/assets/battleUnitItem.fxml`

Esta organización facilita la carga de recursos y evita errores de ruta.

- Integración en la interfaz gráfica

Los iconos se han incorporado a la interfaz mediante componentes JavaFX como:

ImageView para mostrar imágenes

Label para nombres y estadísticas

HBox y VBox para organizar elementos visuales

El archivo battleUnitItem.fxml actúa como plantilla visual para cada unidad mostrada en la vista de batalla, permitiendo reutilizar el mismo diseño para diferentes tipos de unidades.

- Asignación de iconos según el tipo de unidad

Cada unidad del juego se muestra con su icono correspondiente mediante un sistema de asignación definido en los controladores de la interfaz. Esto permite que cada tipo de unidad tenga una representación visual coherente y fácilmente identificable.

- Integración en la lógica de presentación

Los controladores de la interfaz (como los encargados de las batallas o la gestión de unidades) utilizan estos recursos gráficos para mostrar información visual al usuario, manteniendo la coherencia estética y funcional del proyecto.

6. Reflexión y mejoras

6.1. Qué dificultades han tenido y cómo las han superado

- Problemas de configuración del entorno

El equipo se encontró con incompatibilidades entre versiones de Java, configuraciones duplicadas y errores derivados del uso de JRE en lugar de JDK. Estas dificultades se resolvieron unificando la versión de Java, corrigiendo los paquetes y eliminando configuraciones incorrectas.

- Desorganización inicial del proyecto

Las clases estaban dispersas y sin una estructura clara, lo que dificultaba la comprensión y el mantenimiento del código.

Se reorganizó el proyecto en paquetes coherentes (Buildings, Units, Logic, Interfaces, Exceptions, Save_Load), facilitando la navegación y el trabajo en equipo.

- Errores en la lógica del juego

La amenaza no se actualizaba, las batallas automáticas no se ejecutaban y algunos valores no se inicializaban correctamente.

Se revisaron los controladores, se ajustaron los métodos de inicialización y se corrigieron los temporizadores para garantizar el funcionamiento correcto.

- Problemas en la interfaz gráfica

Faltaban archivos FXML, algunos iconos no se cargaban y la información no se actualizaba al abrir ciertas vistas.

Se añadieron los componentes necesarios, se corrigieron rutas y se implementaron actualizaciones en tiempo real para mostrar datos coherentes.

- Falta de persistencia de datos

El juego no guardaba ni cargaba partidas.

Se implementó un sistema de persistencia mediante JDBC y se añadieron opciones en el menú para gestionar el progreso.

6.2. Mejoras futuras o nuevas funcionalidades

- Sistema de sonido y música ambiental

Actualmente solo se utilizan recursos visuales. Una mejora futura sería añadir efectos de sonido y música para aumentar la inmersión del jugador.

- Animaciones en la interfaz

La interfaz podría enriquecerse con animaciones suaves para transiciones, botones y eventos de batalla.

- Mejoras en la IA enemiga

El sistema de generación de enemigos podría ampliarse para crear comportamientos más dinámicos y adaptativos.

- Ampliación del sistema de recursos

Se podrían añadir nuevos tipos de recursos, edificios o tecnologías para aumentar la profundidad del juego.

- Historial de batallas y estadísticas avanzadas

Una sección dedicada a estadísticas permitiría al jugador consultar su progreso, victorias, derrotas y evolución de la amenaza.

- Modo campaña o misiones

Implementar objetivos progresivos o niveles permitiría transformar el juego en una experiencia más estructurada.